CS527 – Lab 9: OpenMP Report (Tasks, Scheduling, Multiple Reductions)

Name: Soudip Das · Roll no.: 2026CSM1037

1. Setup and method

Item	Value
Language	C++ (standard C++ with OpenMP)
Compiler	g++ from MSYS2 UCRT64 (Windows)
Compile command	`g++ -fopenmp -O2 -Wall taskN_xxx.cpp -o taskN` (no warnings)
Machine	[CPU model : Intel(R) Core(TM) Ultra 5 225H (1.70 GHz)], [RAM : 32.0 GB (31.4 GB usable)]. OpenMP reports 14 hardware threads (omp_get_max_threads() = 14)
Timing	`omp_get_wtime()`
Setting the number of threads	**In code, with `omp_set_num_threads(n)`**. Each program loops over 1, 2, 4 and 8 threads itself
Correctness	Every parallel result is compared with the serial result. All Correct / Sorted columns are **YES**

Speedup = T_serial / T_parallel. **Efficiency** = speedup / number of threads.

Measurement notes. The timer on this Windows setup ticks in steps of about 1 ms, so times of a few milliseconds are coarse (Tasks 1 and 3 repeat short jobs and report the average). Each configuration was measured once per program run, and single runs vary by about 10%, so small differences should not be over-interpreted.

2. Task 1 – Merge sort with OpenMP tasks

What was done. A serial merge sort (`merge_sort_serial` + `merge`) and a parallel version where the sorting of the left and right halves are two tasks (`#pragma omp task`). The parent waits with `#pragma omp taskwait` before merging. Sub-arrays smaller than the cutoff (2048 elements) are sorted serially. A temporary array is allocated once before sorting; every call works on its own range `left..right`, so tasks never touch the same elements. The recursion is started inside `#pragma omp parallel` + `#pragma omp single`. Every parallel result was checked to be sorted and identical to the serial output (**YES** in all rows).

Part 1 – serial vs parallel (cutoff 2048)

N = 10^5 (serial 6.280 ms)

Threads	Parallel (ms)	Speedup	Efficiency	Sorted + matches serial
1	6.100	1.03	1.03	YES
2	3.580	1.75	0.88	YES
4	2.720	2.31	0.58	YES
8	2.100	2.99	0.37	YES

N = 10^6 (serial 73.400 ms)

Threads	Parallel (ms)	Speedup	Efficiency	Sorted + matches serial
1	75.600	0.97	0.97	YES
2	41.000	1.79	0.90	YES
4	28.800	2.55	0.64	YES
8	19.600	3.74	0.47	YES

N = 10^7 (serial 842.000 ms)

Threads	Parallel (ms)	Speedup	Efficiency	Sorted + matches serial
1	842.000	1.00	1.00	YES

Threads	Parallel (ms)	Speedup	Efficiency	Sorted + matches serial
2	449.000	1.88	0.94	YES
4	303.000	2.78	0.69	YES
8	199.000	4.23	0.53	YES

Part 2 – effect of the cutoff (N = 10^6, serial 74.000 ms). Each cell is parallel time in ms (speedup). All results correct.

Cutoff	1 thread	2 threads	4 threads	8 threads
4	108.667 (0.68)	141.000 (0.52)	200.667 (0.37)	281.667 (0.26)
32	80.333 (0.92)	57.333 (1.29)	46.667 (1.59)	47.000 (1.57)
256	77.333 (0.96)	41.333 (1.79)	30.333 (2.44)	21.667 (3.42)
2048	74.000 (1.00)	41.000 (1.80)	29.333 (2.52)	20.667 (3.58)
16384	74.667 (0.99)	39.667 (1.87)	31.000 (2.39)	19.333 (3.83)
131072	76.333 (0.97)	39.333 (1.88)	30.667 (2.41)	20.000 (3.70)
500000	74.000 (1.00)	40.000 (1.85)	41.333 (1.79)	39.000 (1.90)

Explanation of the speedup. The speedup grows with the array size (2.99, 3.74 and 4.23 on 8 threads for N = 10^5, 10^6, 10^7) but efficiency falls as threads are added (0.53 at 8 threads for N = 10^7). With one thread the parallel code is as fast as the serial code, so the task overhead at cutoff 2048 is negligible. The limit comes from the merging: the last merge is done by one thread on the whole array, the next level by two threads, then four, so the top levels cannot use all 8 threads (a rough estimate of the best possible speedup on 8 threads is about 5), and merging also needs a lot of memory bandwidth.

Q: What is the difference between OpenMP tasks and `parallel for`? `parallel for` divides the iterations of a loop whose length is known when the loop starts, and the threads share them according to the schedule. A task is a separate piece of work created while the program runs; any thread in the team can run it later. Tasks therefore suit recursion and irregular work such as merge sort, where the number of pieces is not known in advance.

Q: Why is `taskwait` required before merging? The merge needs both halves to be completely sorted. Tasks run independently and possibly later than the point where they are created, so without `taskwait` the parent could merge halves that are still being sorted, giving a wrong result (a data race on the array).

Q: How does the cutoff threshold affect performance? Table Part 2 shows it. A very small cutoff (4) creates hundreds of thousands of tiny tasks, and the overhead makes the program slower than serial (0.26 on 8 threads), and slower with more threads. Cutoffs from 256 to 131072 all perform well (speedup 3.4 to 3.8 on 8 threads), so the exact value is not critical in that range. A very large cutoff (500000) creates only two tasks, so at most two threads work and the speedup stays near 1.9 whatever the thread count.

Q: Why can parallel merge sort become slower than serial merge sort for small arrays? Starting the thread team and creating and scheduling tasks has a fixed cost, and a small array is sorted in less time than that cost. In our runs with cutoff 2048 the parallel sort was not slower than serial even at N = 10^5 (1.03 on 1 thread and 2.99 on 8), but the gain was small (efficiency 0.37 at 8 threads), and with a small cutoff it was slower than serial even for N = 10^6.

Q: How does task creation overhead affect scalability? Every task has to be created, put in the shared task queue and taken out again, and the more threads there are, the more they compete for that queue. This is visible for cutoff 4, where the time increases from 108.7 ms to 281.7 ms as the threads go from 1 to 8. The cutoff keeps the number of tasks small, but the speedup is still limited because of the merges at the top of the recursion, as explained above.

3. Task 2 – Static, dynamic and guided scheduling

What was done. Each element is updated `work[i]` times (`x = x * 1.0000001 + 0.0000001`), so a larger `work[i]` means a slower iteration. Three workloads were used: **uniform** (same work for all elements), **uneven sawtooth** (work grows from 0 to a maximum and restarts every 1000 elements, as in the sample code) and **uneven ramp** (work grows steadily with i, so the last elements are the slowest). The sample's work of `(i % 1000) * 100` would need about 5 x 10^11 repetitions at N = 10^7 (many minutes), so the program uses about 250 million repetitions in total for every N. The parallel versions are `schedule(static)`, `schedule(dynamic, 100)` and `schedule(guided)`. Each parallel result was identical to the serial result (**Correct = YES** in all rows). Each cell is **time (s) / speedup / efficiency**.

N = 10^7, uniform workload (serial 0.0630 s)

Threads	Static	Dynamic (100)	Guided
1	0.0660 / 0.95 / 0.95	0.0650 / 0.97 / 0.97	0.0650 / 0.97 / 0.97
2	0.0370 / 1.70 / 0.85	0.0370 / 1.70 / 0.85	0.0320 / 1.97 / 0.98
4	0.0210 / 3.00 / 0.75	0.0210 / 3.00 / 0.75	0.0180 / 3.50 / 0.88
8	0.0150 / 4.20 / 0.53	0.0130 / 4.85 / 0.61	0.0110 / 5.73 / 0.72

N = 10^7, uneven workload (sawtooth) (serial 0.0860 s)

Threads	Static	Dynamic (100)	Guided
1	0.0910 / 0.95 / 0.95	0.0910 / 0.95 / 0.95	0.0890 / 0.97 / 0.97
2	0.0480 / 1.79 / 0.90	0.0470 / 1.83 / 0.91	0.0450 / 1.91 / 0.96
4	0.0260 / 3.31 / 0.83	0.0260 / 3.31 / 0.83	0.0230 / 3.74 / 0.93
8	0.0150 / 5.73 / 0.72	0.0160 / 5.38 / 0.67	0.0130 / 6.62 / 0.83

N = 10^7, uneven workload (ramp) (serial 0.0860 s)

Threads	Static	Dynamic (100)	Guided
1	0.0890 / 0.97 / 0.97	0.0890 / 0.97 / 0.97	0.0910 / 0.95 / 0.95
2	0.0690 / 1.25 / 0.62	0.0470 / 1.83 / 0.91	0.0420 / 2.05 / 1.02
4	0.0440 / 1.95 / 0.49	0.0260 / 3.31 / 0.83	0.0200 / 4.30 / 1.08
8	0.0290 / 2.97 / 0.37	0.0160 / 5.37 / 0.67	0.0120 / 7.17 / 0.90

N = 10^6, ramp workload (serial about 0.255 s)

[CHECK before submitting: the pasted output of the program was cut, so the heading of this table is missing. From the speedups the serial time is about 0.255 s, and the pattern of the static column (1.23, 1.88, 3.48) and the dynamic times match the ramp workload at N = 10^6. Please confirm from your full output and delete this note.]

Threads	Static	Dynamic (100)	Guided
1	0.2550 / 1.00 / 1.00	0.2510 / 1.01 / 1.01	0.2560 / 0.99 / 0.99
2	0.2070 / 1.23 / 0.61	0.1260 / 2.02 / 1.01	0.1290 / 1.97 / 0.98
4	0.1350 / 1.88 / 0.47	0.0660 / 3.85 / 0.96	0.0660 / 3.85 / 0.96
8	0.0730 / 3.48 / 0.43	0.0340 / 7.47 / 0.93	0.0330 / 7.70 / 0.96

Tables still to be filled in from your program output (these were not in the pasted log; the program prints them, so re-run `./task2_schedule > task2_output.txt` and copy them):

- N = 10^5: uniform (serial printed as 0.3190 s, rows missing), sawtooth and ramp
- N = 10^6: uniform and sawtooth

[paste the missing tables here]

Chunk size of `schedule(dynamic, chunk)` (N = 10^6, serial 0.2560 s). Each cell is time (s) / speedup. All results correct.

Uniform workload

Chunk	4 threads	8 threads
1	0.1050 / 2.44	0.0550 / 4.65
10	0.0680 / 3.76	0.0340 / 7.53
100	0.0660 / 3.88	0.0330 / 7.76
1000	0.0650 / 3.94	0.0340 / 7.53
10000	0.0650 / 3.94	0.0380 / 6.74
100000	0.0780 / 3.28	0.0520 / 4.92

Ramp workload

Chunk	4 threads	8 threads
1	0.1060 / 2.42	0.0600 / 4.27
10	0.0680 / 3.76	0.0350 / 7.31

Chunk	4 threads	8 threads
100	0.0660 / 3.88	0.0340 / 7.53
1000	0.0670 / 3.82	0.0340 / 7.53
10000	0.0680 / 3.76	0.0360 / 7.11
100000	0.0890 / 2.88	0.0610 / 4.20

Explanation of the speedup. For the uneven ramp workload the schedule makes a large difference: on 8 threads `static` reaches only 2.97 (efficiency 0.37 at N = 10^7), `dynamic` 5.37 and `guided` 7.17. With the ramp, a static block for the last thread contains much more work than the first, which limits `static` to at most 1.33, 2.29 and 4.27 on 2, 4 and 8 threads (T^2 / (2T - 1)); the measured values (1.25, 1.95, 2.97) are close to these limits. For the uniform and sawtooth workloads the three schedules are close (4.2 to 5.7 and 5.4 to 6.6 on 8 threads), with `guided` slightly ahead.

Q: What is the difference between static, dynamic and guided scheduling? `static` splits the iterations into equal blocks before the loop starts, one block per thread, so it has no scheduling cost during the loop but cannot react to unequal work. `dynamic, chunk` hands out chunks of `chunk` iterations on request: a thread that finishes asks for the next chunk, so it balances uneven work but each request costs time. `guided` is like dynamic, but the chunks start large and get smaller (down to a minimum), so it makes fewer requests than dynamic with a small chunk and still balances the end of the loop.

Q: Which schedule is best for uniform workloads, and for uneven workloads? In theory `static` is the best for uniform work, because there is nothing to balance and it has no overhead, provided all threads run at the same speed. In our uniform run at N = 10^7 the schedules were close and `guided` was in fact slightly the best (5.73, against 4.85 for dynamic and 4.20 for static on 8 threads), but those runs take only 11 to 15 ms, close to the 1 ms timer resolution, so the differences are small. For uneven work, `dynamic` and `guided` are clearly better than `static` when the heavy iterations are grouped together (ramp: 7.17 for guided and 5.37 for dynamic against 2.97 for static on 8 threads). The sawtooth workload is an exception: it repeats every 1000 elements, so every static block contains many complete teeth and is already balanced (5.73 for static).

Q: Why can dynamic scheduling introduce additional overhead? Every time a thread finishes a chunk it has to get the next one from a shared place, and this must be done safely so that two threads never get the same chunk. This costs time, and the cost is paid

once per chunk. With chunk size 1 and 10^6 iterations there are 10^6 such requests: on 8 threads the uniform loop took 0.0550 s with chunk 1 against 0.0330 s with chunk 100.

Q: How does the chunk size affect performance? A tiny chunk means many requests (chunk 1 is 1.6 to 1.7 times slower than chunk 100 on both thread counts). A huge chunk means few, big pieces, which cannot be balanced: with chunk 100000 there are only 10 chunks for 8 threads, so some threads get two chunks and the others one (time about 2 x 0.0256 s = 0.051 s, as measured: 0.0520 s), and with 4 threads some get three chunks (0.0780 s). In between, chunk sizes from 10 to 1000 are all good (speedup 7.3 to 7.8 on 8 threads), so the best chunk is a medium one.

4. Task 3 – Counting even and odd elements

What was done. The array holds random integers. The serial loop counts `even` and `odd`. The parallel versions are (1) `#pragma omp parallel for reduction(+:even, odd)` and (2) a version where both shared counters are updated with `#pragma omp atomic`. In every run the parallel counts equal the serial counts and even + odd = N (**Correct = YES**). Counts (even / odd): N = 10^5: 49875 / 50125; 10^6: 500338 / 499662; 10^7: 5000261 / 4999739; 10^8: 49993431 / 50006569. Each cell is **time (ms) / speedup / efficiency**, averaged over repetitions.

N = 10^5 (serial 0.012 ms)

Threads	Reduction	Atomic
1	0.294 / 0.04 / 0.04	0.555 / 0.02 / 0.02
2	0.171 / 0.07 / 0.04	1.434 / 0.01 / 0.00
4	0.163 / 0.07 / 0.02	1.633 / 0.01 / 0.00
8	0.109 / 0.11 / 0.01	2.028 / 0.01 / 0.00

N = 10^6 (serial 0.120 ms)

Threads	Reduction	Atomic
1	2.820 / 0.04 / 0.04	5.540 / 0.02 / 0.02
2	1.590 / 0.08 / 0.04	13.170 / 0.01 / 0.00

Threads	Reduction	Atomic
4	1.070 / 0.11 / 0.03	16.140 / 0.01 / 0.00
8	0.810 / 0.15 / 0.02	16.780 / 0.01 / 0.00

N = 10^7 (serial 7.300 ms)

Threads	Reduction	Atomic
1	29.500 / 0.25 / 0.25	56.900 / 0.13 / 0.13
2	15.700 / 0.46 / 0.23	143.500 / 0.05 / 0.03
4	10.700 / 0.68 / 0.17	165.600 / 0.04 / 0.01
8	6.200 / 1.18 / 0.15	169.800 / 0.04 / 0.01

N = 10^8 (serial 285.000 ms)

Threads	Reduction	Atomic
1	293.000 / 0.97 / 0.97	574.000 / 0.50 / 0.50
2	157.000 / 1.82 / 0.91	1462.000 / 0.19 / 0.10
4	87.000 / 3.28 / 0.82	1635.000 / 0.17 / 0.04
8	66.000 / 4.32 / 0.54	1908.000 / 0.15 / 0.02

Measurement note about the serial baseline. The serial times for N up to 10^7 are suspiciously small: they correspond to only 0.12 to 0.73 ns per element, while the serial loop at N = 10^8 and every 1-thread reduction run take about 2.9 ns per element. The probable reason is that the compiler optimised the repeated calls of the serial loop in the timing loop (the same loop is repeated many times on the same data), so the serial baseline is unreliable for N up to 10^7, and the speedups below 1 for these sizes mostly reflect that. The N = 10^8 row is a single, unrepeated run, so its baseline is reliable. A fairer view for the small sizes is the speedup of the reduction version relative to its own 1-thread time (T1 / Tn):

N	1 thread	2 threads	4 threads	8 threads
10^5	1.00	1.72	1.80	2.70
10^6	1.00	1.77	2.64	3.48
10^7	1.00	1.88	2.76	4.76
10^8	1.00	1.87	3.37	4.44

Explanation of the speedup. The reduction version scales with the number of threads (4.3 times faster than serial at N = 10^8 on 8 threads, efficiency 0.54, and 3.5 to 4.8 times faster than its own 1-thread run for N from 10^6 to 10^8). The atomic version is slower than serial at every size and gets slower as threads are added (N = 10^8: 574 ms on 1 thread, 1908 ms on 8 threads), because all threads update the same two counters and have to wait for each other. At 8 threads the reduction is about 29 times faster than the atomic version at N = 10^8 (66 ms against 1908 ms).

Q: How can multiple variables be included in a reduction clause? List them in the same clause separated by commas: `reduction(+:even, odd)`. Each thread gets its own private copy of every listed variable, and OpenMP combines each of them with the given operator at the end of the loop. A different operator needs its own clause, for example `reduction(+:sum) reduction(max:m)`.

Q: Why is the reduction approach generally faster than using atomic updates? With `reduction` each thread counts in its own private variables, so there is no sharing, no waiting and no communication during the loop, and the private values are combined only once at the end. With `atomic` every element makes the thread update a shared counter, and all threads fight for the same two memory locations, so the updates are effectively one at a time and the cache line holding the counters moves between the cores. Even with 1 thread the atomic version was about twice as slow (574 ms against 293 ms at N = 10^8), because an atomic update is more expensive than a normal one.

Q: Why must the sum of the even and odd counts equal N? Every element is either even or odd (never both, never neither), and each element is visited exactly once by exactly one thread, so each is counted exactly once. The reduction adds all the private counters, so nothing is lost. The program checks `even + odd == N` for every run, and it was true for every version and every size.

Q: Does increasing the number of threads always improve performance? No. For the reduction version the time dropped with every added thread, but far from linearly (efficiency 0.54 at 8 threads for N = 10^8), because this loop does very little work per element and is limited by memory speed. For small arrays the gain is small in absolute terms (0.294 ms to 0.109 ms at N = 10^5) because the fixed cost of the parallel region is comparable to the work. For the atomic version more threads make it much slower. In general, more threads only help if there is enough independent work and little sharing.

5. Overall conclusions

- Task-based merge sort gives a speedup of 3 to 4.2 on 8 threads, limited by the merges at the top of the recursion. The cutoff matters: too small makes it slower than serial, too large leaves threads idle, and a wide middle range works well.
- The loop schedule matters only when the work per iteration is uneven in a way that groups the heavy iterations. `static` was clearly worst for the ramp workload, and `dynamic` and `guided` fixed it. Chunk sizes that are too small or too large both hurt.
- Updating shared counters (`atomic`) is correct but slower than serial and gets worse with more threads. A reduction, which gives every thread private counters, is correct and scales.